Homework set 3

Please **submit this Jupyter notebook through Canvas** no later than **Monday December 2**. **Submit the notebook file with your answers (as .ipynb file) and a pdf printout. The pdf version can be used by the teachers to provide feedback. On canvas there are hints about creating a nice pdf version.**

Before you hand in, please make sure the notebook runs, by running "Restart kernel and run all cells..." from the Kernel menu.

Homework is in **groups of two**, and you are expected to hand in original work. Work that is copied from another group will not be accepted.

Exercise 0

Write down the names + student ID of the people in your group.

Format: Name [ID]

1. Kushnava Singha [15663388]
2. Shania Sinha [14379031]

Run the following cell to import NumPy, Matplotlib. If anything else is needed you can import this yourself.

In [1]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

Exercise 1: Nonlinear least squares

This exercise is about the Gauss-Newton method, and the Levenberg-Marquardt method, which are discussed in section 6.6 of Heath. Please read this section before making this homework set. **In this exercise set the Levenberg-Marquardt method is a little different from the one in Heath. The first equation in subsection 6.6.2 is replaced by**

$$\left(\boldsymbol{J}^T(\boldsymbol{x}_k)\boldsymbol{J}(\boldsymbol{x}_k) + \mu_k \,\mathrm{Diagonal}\left(\boldsymbol{J}^T(\boldsymbol{x}_k)\boldsymbol{J}(\boldsymbol{x}_k)\right)\right)\boldsymbol{s}_k = -\boldsymbol{J}^T(\boldsymbol{x}_k)\,\boldsymbol{r}(\boldsymbol{x}_k)$$

Here $\mathrm{Diagonal}(\boldsymbol{B})$ denotes the diagonal part of $\boldsymbol{B}$. So $\mathrm{Diagonal}(\boldsymbol{B})$ has the same shape as $\boldsymbol{B}$ and identical entries on the diagonal and it has zero off-diagonal entries.

The algorithm for Levenberg-Marquardt, with μ_k constant (denoted μ here), is then $\boldsymbol{x}_0 = \text{initial guess}$ $\mu = \text{constant}$ \bf for $k = 0, 1, 2, \ldots$ $\boldsymbol{A} = \boldsymbol{J}_f(\boldsymbol{x}_k)$ solve $\boldsymbol{s}_k$ from $(\boldsymbol{A}^T\boldsymbol{A} + \mu\,\mathrm{Diagonal}(\boldsymbol{A}^T\boldsymbol{A}))\boldsymbol{s}_k = -\boldsymbol{A}^T\boldsymbol{r}(\boldsymbol{x}_k)$ $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \boldsymbol{s}_k$ \bf end

This reduces to the Gauss-Newton method if $\mu = 0$.

(a)

Implement the Levenberg-Marquardt method with constant μ using a suitable stopping criterion. Make it such that the user can specify the value of the tolerance in the stopping criterion via a parameter `tol` and the maximum number of iterations via a parameter `maxIter`. In the implementation you can use library functions for linear algebra operations.

In [2]:
```python
def levenberg_marquardt(func, jacobian, t, x0, y, tol=1e-8, maxIter=100, mu=0.1):
    x = x0 # Initial guess

    for iters in range(maxIter):
        r = y - func(t, x)     # Residual
        J = jacobian(t, x)     # Jacobian of the function

        gradient = J.T @ r     # Gradient of the function
        H = J.T @ J            # Hessian of the function, A^t*A

        I = np.eye(H.shape[0]) # Identity matrix
        d_x = np.linalg.solve(H + mu * I, gradient)

        x_new = x + d_x        # Update x

        # Check if delta is small enough, if so return the solution
        if np.linalg.norm(d_x) < tol:
            return x_new, iters+1
        x = x_new

    return x, maxIter
```

(b)

The time course of drug concentration y in the bloodstream is well described by

$$y = c_1 t e^{c_2 t}, \tag{1}$$

where t denotes time after the drug was administered. The characteristics of the model are a quick rise as the drug enters the bloodstream, followed by slow exponential decay. The half-life of the drug is the time from the peak concentration to the time it drops to half that level. The measured level of the drug norfluoxetine in a patient's bloodstream at whole hours after it was administered is given in the following data:

In [3]:
```python
# time in hours (t)
hour = np.array( [ 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0 ] )

# concentration in ng/ml (y)
concentration = np.array( [ 8.0, 12.3, 15.5, 16.8, 17.1, 15.8, 15.2, 14.0 ] )
```

Use the Gauss-Newton method to fit this data to the blood concentration model (1).

Also use the Levenberg-Marquardt method with $\mu = 0.1$ to address the same problem.

Which method produces the least number of iterations? N.B. clearly state the starting point.

You are asked to use your own version of Gauss-Newton and Levenberg-Marquardt.

Answer:

After fitting both the models Gauss-Newton and Levenberg-Marquardt to the blood concentration data provided and by using the starting point as $c_1 = 3$ and $c_2 = -0.1$, it is observed that the Gauss-Newton Model converges faster than the Levenberg-Marquardt Model. Levenberg-Marquardt takes 10 iterations to converge for this particular starting point instead of the 7 iterations taken by the Gauss-Newton Model.

This could be attributed to the fact that Gauss-Newton method solves the problem without using the regularization term (μI). This means that if the starting point $(3, -0.1)$ is close to the optimal point $(9.8, -0.22)$, the Gauss-Newton Method converges faster. However, if the optimal point is far away from the initial guess (as we see in part (c)), the Gauss-Newton method diverges.

In [4]:
```python
def gauss_newton(func, jacobian, t, x0, y, tol=1e-6, maxIter=100):
    x = x0                  # Initial guess

    for iter in range(maxIter):
        r = y - func(t, x)  # Residual
        J = jacobian(t, x)  # Jacobian of the function

        gradient = J.T @ r  # Gradient of the function
        H = J.T @ J         # Hessian of the function, A^t*A

        delta_x = np.linalg.solve(H, gradient)

        x_new = x + delta_x

        if np.linalg.norm(delta_x) < tol:
            return x_new, iter+1

        x = x_new
    return x, maxIter
```

In [5]:
```python
# Function: f(c, t) = c1 * t * exp(c2 * t)
def func(t, c):
    return c[0] * t * np.exp(c[1] * t)

# Jacobian of the function defined above
def jacobian(t, c):
    J = np.zeros((len(t), 2))
    J[:, 0] = t * np.exp(c[1] * t)
    J[:, 1] = c[0] * t**2 * np.exp(c[1] * t)
    return J
```

In [6]:
```python
# Initial guess = (3, -0.1)
x0 = np.array([3.0, -0.1])

# Gauss-Newton
gn_result, gn_iterations = gauss_newton(func, jacobian, hour, x0, concentration)
print(f"Gauss-Newton: c1 = {gn_result[0]:.2f}, c2 = {gn_result[1]:.2f}, Iterations = {gn_iterations

# Levenberg-Marquardt
lm_result, lm_iterations = levenberg_marquardt(func, jacobian, hour, x0, concentration, mu=0.1)
print(f"Levenberg-Marquardt: c1 = {lm_result[0]:.2f}, c2 = {lm_result[1]:.2f}, Iterations = {lm_ite
```

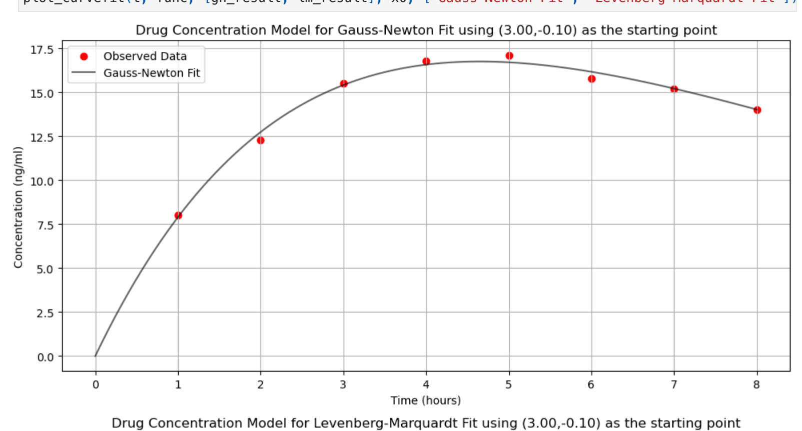
```
Gauss-Newton: c1 = 9.80, c2 = -0.22, Iterations = 7
Levenberg-Marquardt: c1 = 9.80, c2 = -0.22, Iterations = 10
```

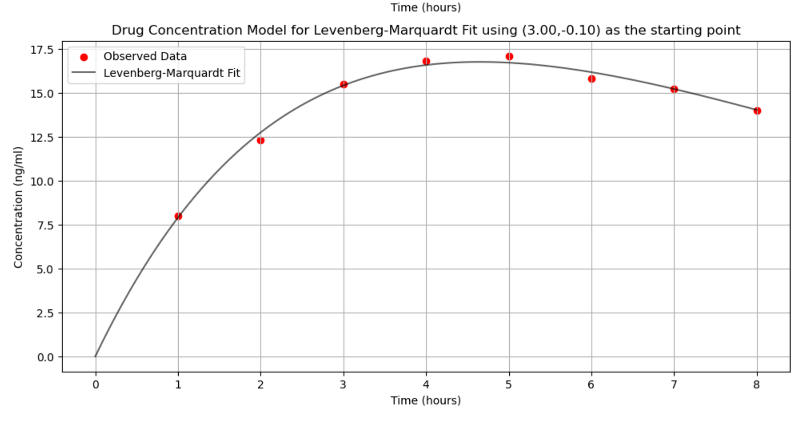
In [7]:
```python
# Plot the curve fit
def plot_curvefit(t, func, results, x0, labels):
    fig, axes = plt.subplots(len(results), 1, figsize=(10, 5 * len(results)))

    if len(results) == 1:
        axes = [axes]

    for ax, result, label in zip(axes, results, labels):
        ax.scatter(hour, concentration, label="Observed Data", color="red")
        ax.plot(t, func(t, result), label=label, color="black", alpha=0.6)
        ax.set_xlabel("Time (hours)")
        ax.set_ylabel("Concentration (ng/ml)")
        ax.set_title(f"Drug Concentration Model for {label} using ({x0[0]:0.2f},{x0[1]:0.2f}) as th
        ax.legend()
        ax.grid()
    plt.tight_layout()
    plt.show()
```

In [8]:
```python
t = np.linspace(0, 8, 100)
plot_curvefit(t, func, [gn_result, lm_result], x0, ['Gauss-Newton Fit', 'Levenberg-Marquardt Fit'])
```





(c)

Try to find a starting point such that Gauss-Newton does not converge, while Levenberg-Marquardt does.

In [9]:
```python
# Initial guess = (3, -0.4)
x0 = np.array([3.0, -0.4])

# Gauss-Newton
gn_result, gn_iterations = gauss_newton(func, jacobian, hour, x0, concentration)
print(f" Gauss-Newton: c1 = {gn_result[0]:.2f}, c2 = {gn_result[1]:.2f}, Iterations = {gn_iteration

# Levenberg-Marquardt
lm_result, lm_iterations = levenberg_marquardt(func, jacobian, hour, x0, concentration, mu=0.1)
print(f" Levenberg-Marquardt: c1 = {lm_result[0]:.2f}, c2 = {lm_result[1]:.2f}, Iterations = {lm_it
```

```
Gauss-Newton: c1 = 0.56, c2 = -0.27, Iterations = 100
Levenberg-Marquardt: c1 = 9.80, c2 = -0.22, Iterations = 26
```

In [10]:
```python
# Table of results
results1 = pd.DataFrame({
    'Method': ['Gauss-Newton', 'Levenberg-Marquardt'],
    'c1': [gn_result[0], lm_result[0]],
    'c2': [gn_result[1], lm_result[1]],
    'Iterations': [gn_iterations, lm_iterations],
    'Converges': ['Yes' if gn_iterations < 100 else 'No', 'Yes' if lm_iterations < 100 else 'No']
})

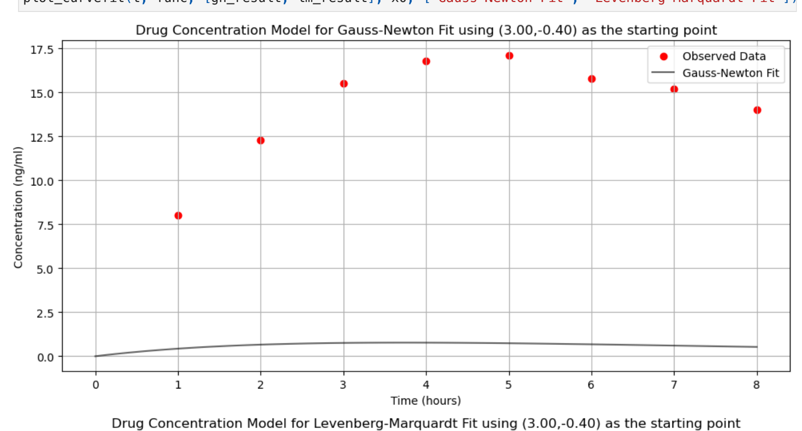
df1 = pd.DataFrame(results1)

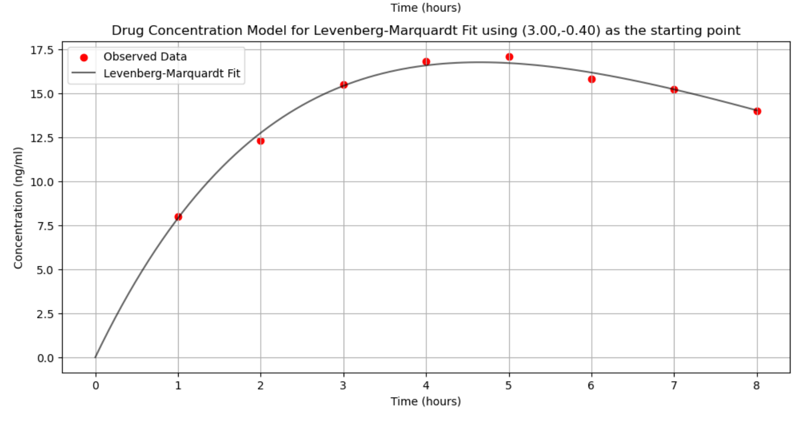
df1
```

Out[10]:

	Method	c1	c2	Iterations	Converges
0	Gauss-Newton	0.562560	-0.267676	100	No
1	Levenberg-Marquardt	9.796928	-0.215087	26	Yes

In [11]:
```python
# Plot the curve fit
plot_curvefit(t, func, [gn_result, lm_result], x0, ['Gauss-Newton Fit', 'Levenberg-Marquardt Fit'])
```





(d)

So far for simplicity, we considered constant μ. However Levenberg-Marquardt is often applied adaptively with a varying μ. A common strategy is to continue to decrease μ by a factor of 10 on each iteration step as long as the residual sum of squared errors is decreased by the step, and if the sum increases, to reject the step and increase μ by a factor of 10.

Implement an adaptive variant of Levenberg-Marquardt using such a strategy for choosing μ.

Compare the performance (iteration number) of the adaptive variant with Gauss-Newton and the previous, non-adaptive variant of Levenberg-Marquardt. Consider a starting point for which Gauss-Newton converged rapidly and a starting point for which Gauss-Newton did not converge, but non-adaptive Levenberg-Marquardt did. Give your answer in a table for clarity, also indicating the starting point and if relevant other parameters.

In [12]:
```python
def adaptive_levenberg_marquardt(func, jacobian, t, x0, y, tol=1e-6, maxIter=100, mu=0.01):
    x = x0              # Initial guess
    prev_sse = np.inf   # Previous sum of squared errors

    for iter in range(maxIter):
        r = y - func(t, x)  # Residual
        J = jacobian(t, x)  # Jacobian of the function

        sse = np.sum(r**2)  # Sum of squared errors

        gradient = J.T @ r  # Gradient of the function
        H = J.T @ J         # Hessian of the function, A^t*A

        I = np.eye(H.shape[0])
        delta_x = np.linalg.solve(H + mu * I, gradient)

        x_new = x + delta_x # Update x

        # Adaptive mu: If the new sum of squared errors is
        # less than the previous sum of squared errors, decrease mu
        if sse < prev_sse:
            mu /= 10
            prev_sse = sse   # Update the previous sum of squared errors
            x = x_new        # Update x
        else:
            mu *= 10         # Otherwise, increase mu

        # Check if delta is small enough, if so return the solution
        if np.linalg.norm(delta_x) < tol:
            return x, iter + 1

    return x, maxIter
```

In [13]:
```python
x0_1 = np.array([3.0, -0.1])

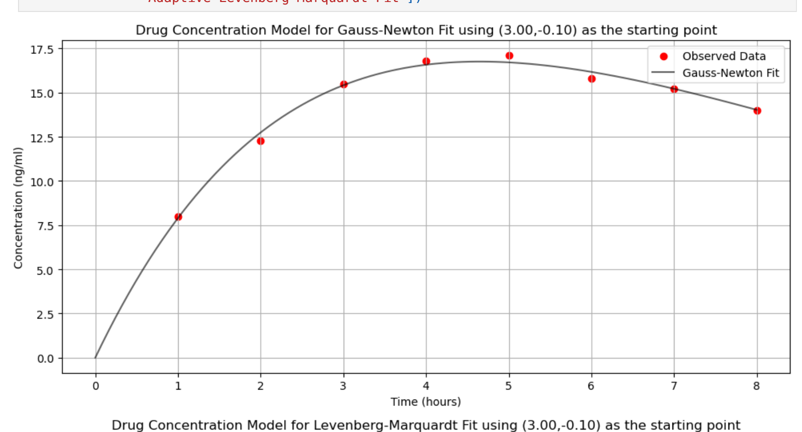
# Gauss-Newton
gn_result_1, gn_iterations_1 = gauss_newton(func, jacobian, hour, x0_1, concentration)
print(f" Gauss-Newton: c1 = {gn_result_1[0]:.2f}, c2 = {gn_result_1[1]:.2f}, Iterations = {gn_itera

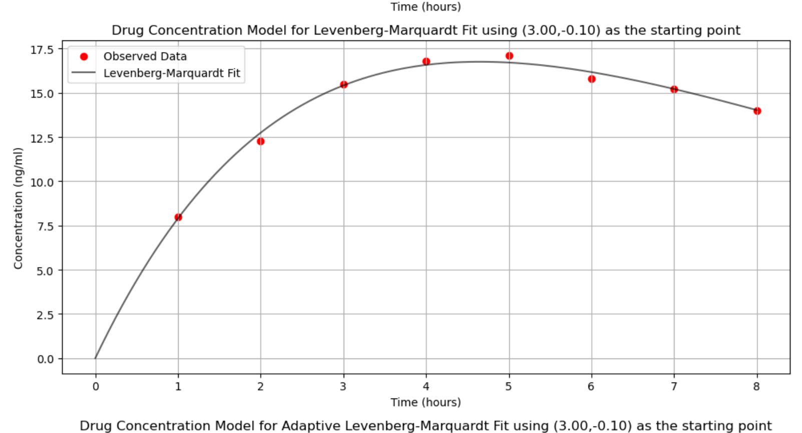
# Levenberg-Marquardt
lm_result_1, lm_iterations_1 = levenberg_marquardt(func, jacobian, hour, x0_1, concentration, mu=0.
print(f" Levenberg-Marquardt: c1 = {lm_result_1[0]:.2f}, c2 = {lm_result_1[1]:.2f}, Iterations = {l

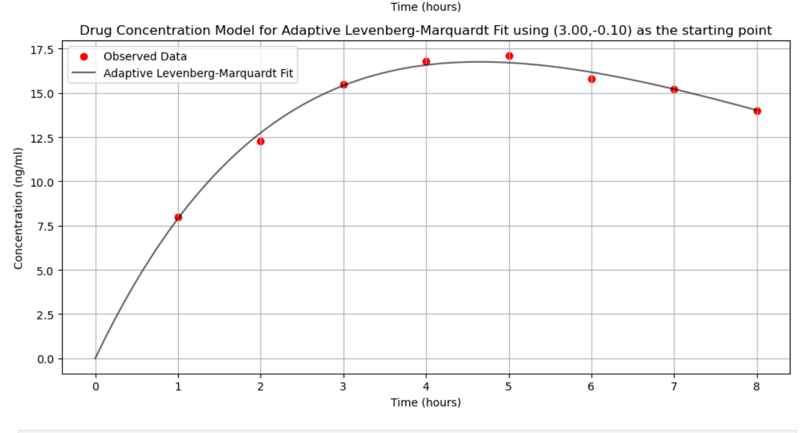
# Adaptive Levenberg-Marquardt
alm_result_1, alm_iterations_1 = adaptive_levenberg_marquardt(func, jacobian, hour, x0_1, concentra
print(f" Adaptive Levenberg-Marquardt: c1 = {alm_result_1[0]:.2f}, c2 = {alm_result_1[1]:.2f}, Iter
```

```
Gauss-Newton: c1 = 9.80, c2 = -0.22, Iterations = 7
Levenberg-Marquardt: c1 = 9.80, c2 = -0.22, Iterations = 10
Adaptive Levenberg-Marquardt: c1 = 9.80, c2 = -0.22, Iterations = 7
```

In [14]:
```python
# Plot the curve fits
plot_curvefit(t, func, [gn_result_1, lm_result_1, alm_result_1], x0_1,
              ['Gauss-Newton Fit',
               'Levenberg-Marquardt Fit',
               'Adaptive Levenberg-Marquardt Fit'])
```







In [15]:
```python
# Routine to go through many possible initial guesses

# # Initial guesses
# x_values = np.linspace(0, 5, 1000)
# y_values = np.linspace(-1, 2, 1000)

# for x in x_values:
#     for y in y_values:
#         x0 = np.array([x, y])
#         try:
#             alm_result, alm_iterations = adaptive_levenberg_marquardt(func, jacobian, hour, x0, c
#             # print(f"x0 = {x0}, Adaptive Levenberg-Marquardt: c1 = {alm_result[0]:.2f}, c2 = {al
#         except np.linalg.LinAlgError as e:
#             if 'Singular matrix' in str(e):
#                 # print("Error for Adaptive Levenberg-Marquardt, x0 = ", x0, "Error: Singular mat
#                 print(end="")
#             continue
#         except RuntimeWarning:
#                 # print("Error for Adaptive Levenberg-Marquardt, x0 = ", x0, "Error: Overflow")
#             print(end="")
#             continue

#         try:
#             gn_result, gn_iterations = gauss_newton(func, jacobian, hour, x0, concentration)
#             # print(f"x0 = {x0}, Gauss-Newton: c1 = {gn_result[0]:.2f}, c2 = {gn_result[1]:.2f},
#         except np.linalg.LinAlgError as e:
#             if 'Singular matrix' in str(e):
#                 # print("Error for Gauss-Newton, x0 = ", x0, "Error: Singular matrix")
#                 print(end="")
#             continue
#         except RuntimeWarning:
#             print(end="")
#             continue

#         if round(alm_result[0], 2) == 9.80 and round(alm_result[1], 2) == -0.22:
#             if round(gn_result[0], 2) != 9.80 and round(gn_result[1], 2) != -0.22:
#                 print(f"Solution: {x0}")
#                 break
```

In [16]:
```python
# Initial guess = (0.35, -0.20)
x0_2 = np.array([0.35035035, -0.2042042])
# x0_2 = np.array([0.01001001, -0.21021021]) # Another possible initial point: (0.01, -0.21)

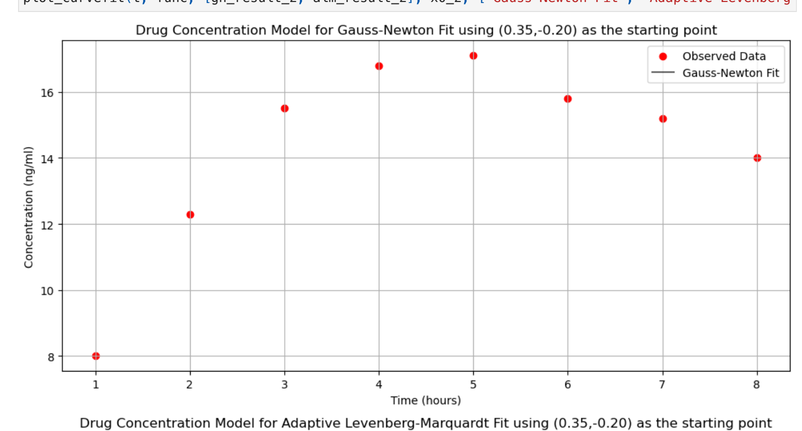
# Gauss-Newton
gn_result_2, gn_iterations_2 = gauss_newton(func, jacobian, hour, x0_2, concentration)
print(f"Gauss-Newton: c1 = {gn_result_2[0]:.2f}, c2 = {gn_result_2[1]:.2f}, Iterations = {gn_iterat

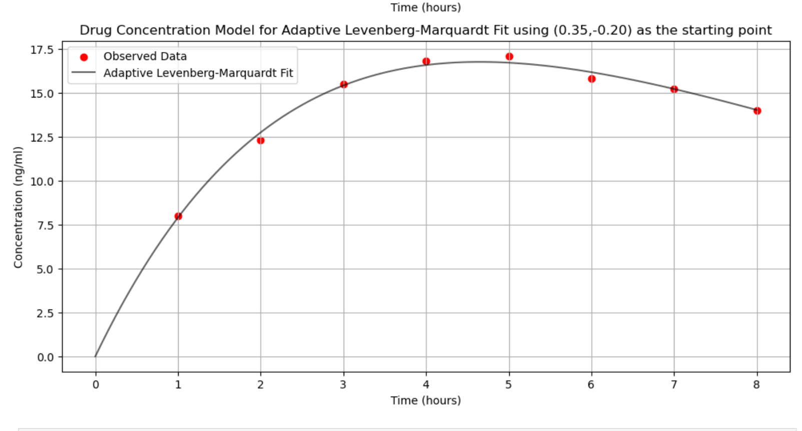
# Levenberg-Marquardt
lm_result_2, lm_iterations_2 = levenberg_marquardt(func, jacobian, hour, x0_2, concentration, mu=0.
print(f"Levenberg-Marquardt: c1 = {lm_result_2[0]:.2f}, c2 = {lm_result_2[1]:.2f}, Iterations = {lm

# Adaptive Levenberg-Marquardt
alm_result_2, alm_iterations_2 = adaptive_levenberg_marquardt(func, jacobian, hour, x0_2, concentra
print(f"Adaptive Levenberg-Marquardt: c1 = {alm_result_2[0]:.2f}, c2 = {alm_result_2[1]:.2f}, Itera
```

```
Gauss-Newton: c1 = nan, c2 = nan, Iterations = 100
Levenberg-Marquardt: c1 = 9.80, c2 = -0.22, Iterations = 26
Adaptive Levenberg-Marquardt: c1 = 9.80, c2 = -0.22, Iterations = 7
```

In [17]:
```python
# Plot the curve fits
plot_curvefit(t, func, [gn_result_2, alm_result_2], x0_2, ['Gauss-Newton Fit', 'Adaptive Levenberg-
```





In [18]:
```python
x0_3 = np.array([3.0, -0.4])

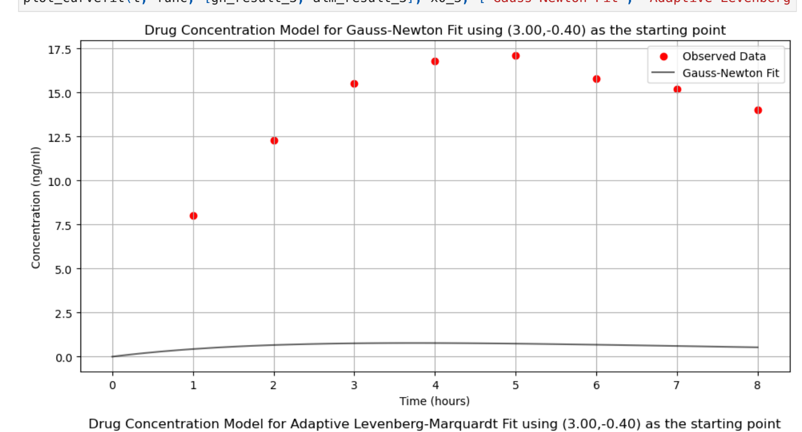
# Gauss-Newton
gn_result_3, gn_iterations_3 = gauss_newton(func, jacobian, hour, x0_3, concentration)
print(f" Gauss-Newton: c1 = {gn_result_3[0]:.2f}, c2 = {gn_result_3[1]:.2f}, Iterations = {gn_itera

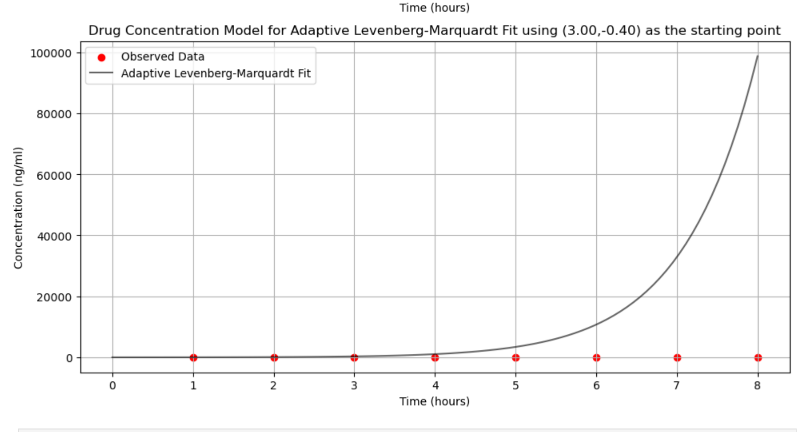
# Levenberg-Marquardt
lm_result_3, lm_iterations_3 = levenberg_marquardt(func, jacobian, hour, x0_3, concentration, mu=0.
print(f" Levenberg-Marquardt: c1 = {lm_result_3[0]:.2f}, c2 = {lm_result_3[1]:.2f}, Iterations = {l

# Adaptive Levenberg-Marquardt
alm_result_3, alm_iterations_3 = adaptive_levenberg_marquardt(func, jacobian, hour, x0_3, concentra
print(f" Adaptive Levenberg-Marquardt: c1 = {alm_result_3[0]:.2f}, c2 = {alm_result_3[1]:.2f}, Iter
```

```
Gauss-Newton: c1 = 0.56, c2 = -0.27, Iterations = 100
Levenberg-Marquardt: c1 = 9.80, c2 = -0.22, Iterations = 26
Adaptive Levenberg-Marquardt: c1 = 5.42, c2 = 0.97, Iterations = 21
```

In [19]:
```python
# Plot the curve fits
plot_curvefit(t, func, [gn_result_3, alm_result_3], x0_3, ['Gauss-Newton Fit', 'Adaptive Levenberg-
```





In [20]:
```python
# Table of results for comparison
methods = ["Gauss-Newton", "Levenberg-Marquardt", "Adaptive Levenberg-Marquardt"]
starting_points = [x0_1, x0_1, x0_1, x0_2, x0_2, x0_2, x0_3, x0_3, x0_3]

results2 = {
    "Method": [methods[0], methods[1], methods[2], methods[0], methods[1], methods[2], methods[0],
    "Starting Point": [f"({x0_1[0]:.3f}, {x0_1[1]:.3f})", f"({x0_1[0]:.3f}, {x0_1[1]:.3f})", f"({x0
    "c": [f"({gn_result_1[0]:.3f}, {gn_result_1[1]:.3f})", f"({lm_result_1[0]:.3f}, {lm_result_1[1]
    "Iterations": [gn_iterations_1, lm_iterations_1, alm_iterations_1, gn_iterations_2, lm_iteratic
    "mu (μ)": ["-", 0.1, 0.1, "-", 0.1, 0.1, "-", 0.1, 0.1],
    "Convergence": ['Yes' if gn_iterations_1 < 100 else 'No', 'Yes' if lm_iterations_1 < 100 else '
}

df2 = pd.DataFrame(results2)

df2
```

Out[20]:

	Method	Starting Point	c	Iterations	mu (μ)	Convergence
0	Gauss-Newton	(3.000, -0.100)	(9.797, -0.215)	7	-	Yes
1	Levenberg-Marquardt	(3.000, -0.100)	(9.797, -0.215)	10	0.1	Yes
2	Adaptive Levenberg-Marquardt	(3.000, -0.100)	(9.797, -0.215)	7	0.1	Yes
3	Gauss-Newton	(0.350, -0.204)	(nan, nan)	100	-	No
4	Levenberg-Marquardt	(0.350, -0.204)	(9.797, -0.215)	10	0.1	Yes
5	Adaptive Levenberg-Marquardt	(0.350, -0.204)	(9.797, -0.215)	7	0.1	Yes
6	Gauss-Newton	(3.000, -0.400)	(0.563, -0.268)	100	-	No
7	Levenberg-Marquardt	(3.000, -0.400)	(9.797, -0.215)	26	0.1	Yes
8	Adaptive Levenberg-Marquardt	(3.000, -0.400)	(5.415, 0.966)	21	0.1	Yes

Answer:

Note that the Adaptive Levenberg-Marquadt method converges for the initial point $(3, -0.4)$ but it does not provide a good fit for the function underlying the given data points for concentration. For the initial point $(0.350, -0.204)$, the Gauss-Newton method encounters numerical overflow due to the computation of excessively large values. This issue can be addressed by implementing better error handling, selecting more suitable initial values, or using alternative approaches to improve numerical stability, such as using higher-precision arithmetic or perhaps a programming language with enhanced numerical capabilities.